

Изучайте Makefiles

С самыми вкусными примерами

[Начать учебник](#)[Макифайл Кулинарная Книга](#)

☆ Star 5,907

Я построил это руководство, потому что я никогда не мог полностью обернуть голову вокруг Makefiles. Они казались наводненными скрытыми правилами и эзотерическими символами, и простые вопросы не давали простых ответов. Чтобы решить эту проблему, я сел на несколько выходных и прочитал все, что мог, о Makefiles. Я скрепил самые важные знания в этом руководстве. Каждая тема имеет краткое описание и самодостаточный пример, что вы можете запустить самостоятельно.

Если вы в основном понимаете Make, подумайте о том, чтобы проверить [поваренную книгу Makefile](#), которая имеет шаблон для проектов среднего размера с достаточным количеством комментариев о том, что делает каждая часть Makefile.

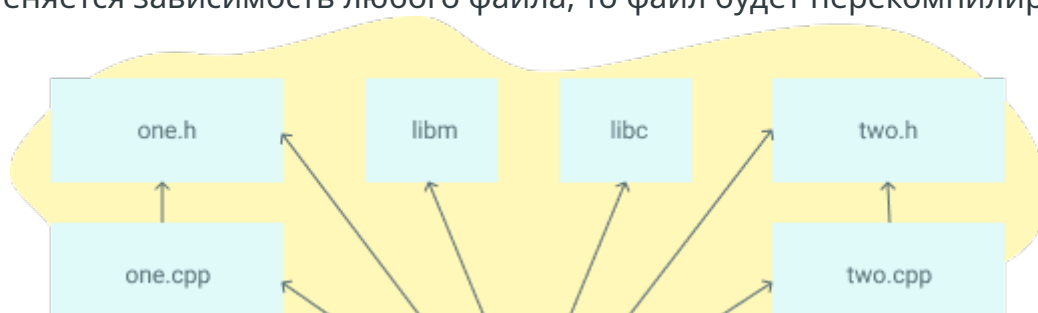
Удачи, и я надеюсь, что вы сможете убить запутанный мир Makefiles!

Начало работы

Почему существуют Makefiles?

Макилфайлы используются, чтобы помочь решить, какие части большой программы необходимо перекомпилировать. В подавляющем большинстве случаев скомпилируются файлы C или C++. Другие языки обычно имеют свои собственные инструменты, которые служат той же цели, что и Make. Make также может быть использована за пределами компиляции, когда вам нужна серия инструкций для запуска в зависимости от того, какие файлы изменились. Этот учебник будет посвящен варианту использования компиляции C/C++.

Вот пример график зависимости, который вы можете построить с помощью Make. Если изменяется зависимость любого файла, то файл будет перекомпилирован:





Какие альтернативы существуют?

Популярными системами альтернативной сборки C/C++ являются [SCons](#), [CMake](#), [Bazel](#) и [Ninja](#). Некоторые редакторы кода, такие как [Microsoft Visual Studio](#), имеют свои собственные инструменты сборки. Для Java есть [Ant](#), [Maven](#) и [Gradle](#). Другие языки, такие как Go, Rust и TypeScript, имеют свои собственные инструменты сборки.

Интерпретируемые языки, такие как Python, Ruby и необработанный Javascript, не требуют аналогов Makefiles. Цель Makefiles состоит в том, чтобы собрать все файлы, которые необходимо скомпилировать, на основе того, какие файлы изменились. Но когда файлы на интерпретируемых языках меняются, ничего не нужно перекомпилировать. Когда программа работает, используется самая последняя версия файла.

Версии и типы Make

Существует множество реализаций Make, но большая часть этого руководства будет работать над любой версией, которую вы используете. Тем не менее, он специально написан для GNU Make, который является стандартной реализацией на Linux и MacOS. Все примеры работают для Make версии 3 и 4, которые почти эквивалентны, кроме некоторых эзотерических различий.

Запуск примеров

Чтобы запустить эти примеры, вам понадобится терминал и «сделать» установлен. Для каждого примера поместите содержимое в файл, называемый `Makefile`, и в этом каталоге запустить команду `make`. Начнем с самого простого из Makefiles:

```
hello:
    echo "Hello, World"
```

Примечание: Макилы **должны** быть отрезаны с помощью TAB, а не с пробелами или `make` будет потерпеть неудачу.

Вот результат запуска вышеприведенного примера:

```
$ make
echo "Hello, World"
Hello, World
```

Вот и все! Если вы немного запутались, вот видео, которое проходит через эти шаги, наряду с описанием основной структуры Makefiles.



Chase Lambert



Watch on

Макайловый синтаксис

Макияж состоит из набора *правил*. Правило вообще выглядит так:

```
targets: prerequisites
  command
  command
  command
```

- *Целями* являются имена файлов, разделенные пространствами. Как правило, есть только одно правило.
- *Команды* представляют собой серию шагов, обычно используемых для создания цели (мишеней). Они *должны начинаться* с символа *вкладки*, а не с пробелов.
- *Предпосылками* являются также имена файлов, разделенные пространствами. Эти файлы должны существовать до запуска команд для цели. Их также называют *зависимыми*.

Суть Make

Давайте начнем с приветливого мирового примера:

```
hello:
  echo "Hello, World"
  echo "This line will print if the file hello does not exist."
```

Здесь уже есть что принять. Давайте разберемся с этим:

- У нас есть одна *цель*, названная `hello`
- Эта цель имеет две *команды*
- Эта цель не имеет *никаких предпосылок*

Мы побежим `make hello`. До тех пор, пока `hello` Файл не существует, команды будут работать. Если `hello` Существует, никакие команды не будут работать.

Важно понимать, что я говорю о `hello` Как в *качестве цели*, так и *file*в файл. Это потому, что они напрямую связаны друг с другом. Как правило, когда цель запущена (он же, когда команды цели запущены), команды создают файл с тем же именем, что и цель. В данном случае, `hello` *Цель* не создает `hello` *Файл*.

Давайте создадим более типичный Makefile - тот, который компилирует один файл C. Но прежде чем мы это сделаем, сделайте файл, названный `blah.c` который имеет следующее содержание:

```
// blah.c
int main() { return 0; }
```

Затем создайте Makefile (называемый `Makefile` , как всегда):

```
blah:
    cc blah.c -o blah
```

На этот раз попробуйте просто бегать `make` . Поскольку нет цели, предоставленной в качестве аргумента `make` Команда, первая цель запущена. В этом случае есть только одна цель (`blah`) В первый раз, когда ты запустишь это, `blah` будет создан. Во второй раз вы увидите `make: 'blah' is up to date` . Это потому, что `blah` Файл уже существует. Но есть проблема: если мы модифицируем `blah.c` и затем бежать `make` , ничего не будет перекомпилировано.

Мы решаем эту проблему, добавляя предпосылку:

```
blah: blah.c
    cc blah.c -o blah
```

Когда мы бежим `make` Опять же, происходит следующий набор шагов:

- Первая цель выбрана, потому что первая цель - цель по умолчанию
- Это имеет предпосылку `blah.c`
- Заставьте решить, должен ли он запустить `blah` Цель. Он будет работать только в том случае, если `blah` не существует, или `blah.c` Это *новее, чем* `blah`

Этот последний шаг имеет решающее значение и является **сутью**. Что он пытается сделать, так это решить, если предпосылки `blah` С тех пор изменились `blah` Был в последний раз составлен. То есть, если `blah.c` изменяется, работает `make` Следует перекомпилировать файл. И наоборот, если `blah.c` не изменился, тогда его не следует перекомпилировать.

Чтобы это произошло, он использует временные метки файловой системы в качестве прокси, чтобы определить, изменилось ли что-то. Это разумная эвристика, потому что временные метки файлов обычно изменяются только в том случае, если файлы Модифицированные. Но важно понимать, что это не всегда так. Например, вы можете изменить файл, а затем изменить измененную метку времени этого файла на что-то старое. Если бы вы это сделали, Make неправильно угадала бы, что файл не изменился и, следовательно, может быть проигнорирован.

Ух, какой рот. **Убедитесь, что вы это понимаете. Это суть Makefiles, и вам может потребоваться несколько минут, чтобы правильно понять.** Играйте с приведенными выше примерами или смотрите видео выше, если все еще сбивает с толку.

Более быстрые примеры

Следующий Makefile в конечном итоге управляет всеми тремя целями. Когда ты бежишь `make` в терминале он построит программу под названием `blah` В нескольких шагах:

- Сделать выбор цели `blah` , потому что первая цель является целью по умолчанию
- `blah` Требуется `blah.o` , так что сделайте поиски для `blah.o` Цель
- `blah.o` Требуется `blah.c` , так что сделайте поиски для `blah.c` Цель
- `blah.c` у него нет зависимостей, поэтому `echo` Команда запущена
- The `cc -c` Команда затем запускается, потому что все `blah.o` Зависимости закончились
- Вершина `cc` Команда запущена, потому что все `blah` Зависимости закончились
- Вот и все: `blah` является компиляционной программой с

```
blah: blah.o
    cc blah.o -o blah # Runs third

blah.o: blah.c
    cc -c blah.c -o blah.o # Runs second

# Typically blah.c would already exist, but I want to limit any addit
blah.c:
    echo "int main() { return 0; }" > blah.c # Runs first
```

Если вы удалите `blah.c` Все три цели будут восстановлены. Если вы редактируете его (и, таким образом, измените метку времени на более новую, чем `blah.o`), первые две цели будут запущены. Если вы бегаєте `touch blah.o` (и, таким образом, изменить временную метку на более новую, чем `blah`), тогда будет бегать только первая цель. Если ничего не менять, ни одна из целей не будет запущена. Попробуй!

Этот следующий пример не делает ничего нового, но, тем не менее, является хорошим дополнительным примером. Он всегда будет выполнять обе цели, потому что `some_file` зависит от `other_file` , который никогда не был создан.

```
some_file: other_file
    echo "This will always run, and runs second"
    touch some_file

other_file:
    echo "This will always run, and runs first"
```

Сделать чистым

`clean` Часто используется в качестве цели, которая удаляет вывод других целей, но это не специальное слово в Make. Вы можете бежать `make` и `make clean` на это, чтобы создать и удалить `some_file` .

Обратите внимание, что `clean` Здесь делают два новых дела:

- Это цель, которая не является первой (по умолчанию), а не обязательным условием. Это означает, что он никогда не будет работать, если вы явно не позвоните `make clean`
- Это не должно быть имя файла. Если у вас есть файл с именем `clean` , эта цель не будет работать, что мы не хотим. Смотреть `.PHONY` позже в этом уроке о том, как это исправить

```
some_file:
    touch some_file
```

```
clean:
    rm -f some_file
```

Переменные

Переменные могут быть только строками. Обычно вы хотите использовать `:=` , но `=` также работает. См. [Переменные Pt 2](#).

Вот пример использования переменных:

```
files := file1 file2
some_file: $(files)
    echo "Look at this variable: " $(files)
    touch some_file
```

```
file1:
    touch file1
file2:
    touch file2
```

```
clean:
    rm -f file1 file2 some_file
```

Одиночные или двойные цитаты не имеют смысла. Это просто символы, которые присваиваются переменной. Цитаты *are* полезны для shell/bash, и вам нужны они в командах, таких как `printf` . В этом примере две команды ведут себя одинаково:

```
a := one two# a is set to the string "one two"
b := 'one two' # Not recommended. b is set to the string "'one two'"
all:
    printf '$a'
    printf $b
```

Эталонные переменные, использующие либо `${}` или `$()`

```
x := dude

all:
    echo $(x)
    echo ${x}

# Bad practice, but works
echo $x
```

Цели

Все цели

Сделаете несколько целей, и вы хотите, чтобы все они были запущены? Сделайте `all` Цель. Поскольку это первое перечисленное правило, оно будет действовать по умолчанию, если `make` называется без указания цели.

```
all: one two three

one:
    touch one
two:
    touch two
three:
    touch three

clean:
    rm -f one two three
```

Множественные цели

Когда есть несколько целей для правила, команды будут выполняться для каждой цели. `$@` [Это автоматическая переменная](#), которая содержит целевое имя.

```
all: f1.o f2.o

f1.o f2.o:
    echo $@
# Equivalent to:
# f1.o:
#   echo f1.o
# f2.o:
#   echo f2.o
```

Автоматические переменные и подстановочные знаки

* Wildcard

Оба `*` и `%` Они называются wildcards в Make, но они означают совершенно разные вещи. `*` ищет файловую систему для сопоставления имен файлов. Я предлагаю, чтобы вы всегда заворачивали его в `wildcard` Функция, потому что в противном случае вы можете попасть в общую ловушку, описанную ниже.

```
# Print out file information about every .c file
print: $(wildcard *.c)
    ls -la $?
```

`*` может быть использован в цели, предпосылках или в `wildcard` Функция.

Опасность: `*` не может быть непосредственно использован в переменных определениях

Опасность: Когда `*` не соответствует никаким файлам, он остается как есть (если только не запускается в `wildcard` функция)

```
thing_wrong := *.o # Don't do this! '*' will not get expanded
thing_right := $(wildcard *.o)
```

```
all: one two three four
```

```
# Fails, because $(thing_wrong) is the string "*.o"
one: $(thing_wrong)
```

```
# Stays as *.o if there are no files that match this pattern :(
two: *.o
```

```
# Works as you would expect! In this case, it does nothing.
three: $(thing_right)
```

```
# Same as rule three
four: $(wildcard *.o)
```

% Wildcard

`%` Это действительно полезно, но несколько сбивает с толку из-за разнообразия ситуаций, в которых он может быть использован.

- При использовании в режиме «соответствия» он сопоставляет один или несколько символов в строке. Этот матч называется стебель.
- При использовании в режиме «замены» он берет стебель, который был сопоставлен, и заменяет его в строке.

- % Чаще всего используется в определениях правил и в некоторых конкретных функциях.

См. эти разделы на примерах его использования:

- [Статические правила шаблона](#)
- [Правила шаблона](#)
- [Замещение струн](#)
- [Директива vpath](#)

Автоматические переменные

Есть много [автоматических переменных](#), но часто появляются только несколько:

```
hey: one two
# Outputs "hey", since this is the target name
echo $@

# Outputs all prerequisites newer than the target
echo $?

# Outputs all prerequisites
echo $^

# Outputs the first prerequisite
echo $<

touch hey

one:
touch one

two:
touch two

clean:
rm -f hey one two
```

Причудливые Правила

Неявные правила

Делайте любовь с компиляцией. И каждый раз, когда он выражает свою любовь, вещи сбивают с толку. Пожалуй, самая запутанная часть Make - это магические/автоматические правила, которые сделаны. Делайте звонки эти «неявные» правила. Я лично не согласен с этим дизайнерским решением, и я не рекомендую использовать их, но они часто используются и, таким образом, полезны для изучения. Вот список неявных правил:

- Составление программы C: `n.o` производится автоматически из `n.c` с командой формы `$(CC) -c $(CPPFLAGS) $(CFLAGS) $^ -o $@`
- Составление программы на C++: `n.o` производится автоматически из `n.cc` или `n.cpp` с командой формы `$(CXX) -c $(CPPFLAGS) $(CXXFLAGS) $^ -o $@`
- Связывание одного объектного файла: `n` производится автоматически из `n.o` запустив команду `$(CC) $(LDFLAGS) $^ $(LOADLIBES) $(LDLIBS) -o $@`

Важными переменными, используемыми неявными правилами, являются:

- `CC` : Программа компиляции программ C; дефолт `cc`
- `CXX` : Программа компиляции программ C++; дефолт `g++`
- `CFLAGS` : Дополнительные флаги, чтобы дать компилятору C
- `CXXFLAGS` : Дополнительные флаги, чтобы дать компилятору C++
- `CPPFLAGS` : Дополнительные флаги для предварительного процессора C
- `LDFLAGS` : Дополнительные флаги, которые можно дать компиляторам, когда они должны вызывать линкер

Давайте посмотрим, как теперь мы можем построить программу C, никогда явно не сообщая Make, как сделать компиляцию:

```
CC = gcc # Flag for implicit rules
CFLAGS = -g # Flag for implicit rules. Turn on debug info

# Implicit rule #1: blah is built via the C linker implicit rule
# Implicit rule #2: blah.o is built via the C compilation implicit rule
blah: blah.o

blah.c:
    echo "int main() { return 0; }" > blah.c

clean:
    rm -f blah*
```

Статические правила шаблона

Статические правила шаблона - это еще один способ меньше писать в Makefile. Вот их синтаксис:

```
targets...: target-pattern: prereq-patterns ...
    commands
```

Суть в том, что данные `target` Соответствует с `target-pattern` (через `%` wildcard). Все, что было сопоставлено, называется стеблем. Затем стебель заменяется в `prereq-pattern`, чтобы генерировать предварительные заказы цели.

Типичный вариант использования - компилировать `.c` Файлы в `.o` Файлы. Вот *ручной способ*:

```
objects = foo.o bar.o all.o
```

```
all: $(objects)
    $(CC) $^ -o all

foo.o: foo.c
    $(CC) -c foo.c -o foo.o

bar.o: bar.c
    $(CC) -c bar.c -o bar.o

all.o: all.c
    $(CC) -c all.c -o all.o

all.c:
    echo "int main() { return 0; }" > all.c

# Note: all.c does not use this rule because Make prioritizes more sp
%.c:
    touch $@

clean:
    rm -f *.c *.o all
```

Вот более *эффективный способ*, используя правило статического рисунка:

```
objects = foo.o bar.o all.o
all: $(objects)
    $(CC) $^ -o all

# Syntax - targets ...: target-pattern: prereq-patterns ...
# In the case of the first target, foo.o, the target-pattern matches
# It then replaces the '%' in prereq-patterns with that stem
$(objects): %.o: %.c
    $(CC) -c $^ -o $@

all.c:
    echo "int main() { return 0; }" > all.c

# Note: all.c does not use this rule because Make prioritizes more sp
%.c:
    touch $@

clean:
    rm -f *.c *.o all
```

Статические правила шаблона и фильтр

Хотя я ввожу [функцию фильтра](#) позже, ее часто используют в статических правилах шаблона, поэтому я упомяну об этом здесь. The `filter` Функция может быть использована в правилах статического шаблона для сопоставления правильных

файлов. В этом примере я составил `.raw` и `.result` расширения.

```
obj_files = foo.result bar.o lose.o
src_files = foo.raw bar.c lose.c

all: $(obj_files)
# Note: PHONY is important here. Without it, implicit rules will try
.PHONY: all

# Ex 1: .o files depend on .c files. Though we don't actually make th
$(filter %.o,$(obj_files)): %.o: %.c
    echo "target: $@ prereq: $<"

# Ex 2: .result files depend on .raw files. Though we don't actually
$(filter %.result,$(obj_files)): %.result: %.raw
    echo "target: $@ prereq: $<"

%.c %.raw:
    touch $@

clean:
    rm -f $(src_files)
```

Правила шаблона

Правила шаблона часто используются, но довольно запутанные. Вы можете смотреть на них как на два способа:

- Способ определить свои собственные неявные правила
- Более простая форма статических правил шаблона

Начнем сначала с примера:

```
# Define a pattern rule that compiles every .c file into a .o file
%.o : %.c
    $(CC) -c $(CFLAGS) $(CPPFLAGS) $< -o $@
```

Правила шаблона содержат «%» в цели. Это «%» соответствует любой непустой строке, и другие персонажи совпадают сами. «%» в предпосылке правила шаблона означает тот же стебель, который соответствовал «%» в цели.

Вот еще один пример:

```
# Define a pattern rule that has no pattern in the prerequisites.
# This just creates empty .c files when needed.
%.c:
    touch $@
```

Правила двойной клорон

Правила двойной толстой кишки используются редко, но позволяют определять несколько правил для одной и той же цели. Если бы это были единичные толстые кишки, было бы напечатано предупреждение, и только второй набор команд работал бы.

```
all: blah
```

```
blah::  
    echo "hello"
```

```
blah::  
    echo "hello again"
```

Команды и исполнение

Командное эхо/заглушивание

Добавить а @ перед командой, чтобы предотвратить его печать
Вы также можете запустить с -s чтобы добавить а @ перед каждой строкой

```
all:  
    @echo "This make line will not be printed"  
    echo "But this will"
```

Командное исполнение

Каждая команда запускается в новой оболочке (или, по крайней мере, эффект как таковой).

```
all:  
    cd ..  
    # The cd above does not affect this line, because each command is €  
    echo `pwd`  
  
    # This cd command affects the next because they are on the same lir  
    cd ../;echo `pwd`  
  
    # Same as above  
    cd ../; \  
    echo `pwd`
```

По умолчанию Shell

Оболочка по умолчанию - это /bin/sh . Вы можете изменить это, изменив переменную SHELL:

```
SHELL=/bin/bash
```

```
cool:
    echo "Hello from bash"
```

Знак двойного доллара

Если вы хотите, чтобы строка имела знак доллара, вы можете использовать `$$` .
Вот как использовать переменную оболочки в `bash` или `sh` .

Обратите внимание на различия между переменными Makefile и переменными Shell в следующем примере.

```
make_var = I am a make variable
all:
    # Same as running "sh_var='I am a shell variable'; echo $sh_var" in
    sh_var='I am a shell variable'; echo $$sh_var

    # Same as running "echo I am a make variable" in the shell
    echo $(make_var)
```

Ошибка обработки с `-k` , `-i` , и `-`

Добавить `-k` при беге делаете, чтобы продолжать бегать даже перед лицом ошибок. Полезно, если вы хотите увидеть все ошибки Make сразу.

Добавить `a -` перед командой для подавления ошибки

Добавить `-i` чтобы это произошло для каждой команды.

```
one:
    # This error will be printed but ignored, and make will continue to
    -false
    touch one
```

Прерывание или убийство сделать

Примечание только: Если вы `ctrl+c` сделать, он удалит новые цели, которые он только что сделал.

Рекурсивное использование Make

Чтобы рекурсивно вызвать макет, используйте специальный `$(MAKE)` вместо того, чтобы `make` потому что он пройдет флаги для вас и сам не будет затронут ими.

```
new_contents = "hello:\n\ttouch inside_file"
all:
    mkdir -p subdir
    printf $(new_contents) | sed -e 's/^ //' > subdir/makefile
    cd subdir && $(MAKE)
```

```
clean:
    rm -rf subdir
```

Экспорт, окружающая среда и рекурсивная марка

Когда Make start start, он автоматически создает переменные Make из всех переменных среды, которые устанавливаются при его исполнении.

```
# Run this with "export shell_env_var='I am an environment variable';
all:
    # Print out the Shell variable
    echo $$shell_env_var

    # Print out the Make variable
    echo $(shell_env_var)
```

The `export` Директива берет переменную и устанавливает ее среду для всех команд оболочки во всех рецептах:

```
shell_env_var=Shell env var, created inside of Make
export shell_env_var
all:
    echo $(shell_env_var)
    echo $$shell_env_var
```

Как таковой, когда вы управляете `make` Команда внутри марки, вы можете использовать `export` Директива, чтобы сделать его доступным для команд-подделки. В этом примере, `cooly` экспортируется таким образом, что макет в `subdir` может использовать его.

```
new_contents = "hello:\n\techo \$$$(cooly)"

all:
    mkdir -p subdir
    printf $(new_contents) | sed -e 's/^ //' > subdir/makefile
    @echo "---MAKEFILE CONTENTS---"
    @cd subdir && cat makefile
    @echo "---END MAKEFILE CONTENTS---"
    cd subdir && $(MAKE)

# Note that variables and exports. They are set/affected globally.
cooly = "The subdirectory can see me!"
export cooly
# This would nullify the line above: unexport cooly

clean:
    rm -rf subdir
```

Вам также нужно экспортировать переменные, чтобы они были запущены в

оболочке.

```
one=this will only work locally
export two=we can run subcommands with this
```

```
all:
    @echo $(one)
    @echo $$one
    @echo $(two)
    @echo $$two
```

`.EXPORT_ALL_VARIABLES` экспортировать все переменные для вас.

```
.EXPORT_ALL_VARIABLES:
new_contents = "hello:\n\techo \$$$(cooly)"
```

```
cooly = "The subdirectory can see me!"
# This would nullify the line above: unexport cooly
```

```
all:
    mkdir -p subdir
    printf $(new_contents) | sed -e 's/^ //' > subdir/makefile
    @echo "---MAKEFILE CONTENTS---"
    @cd subdir && cat makefile
    @echo "---END MAKEFILE CONTENTS---"
    cd subdir && $(MAKE)
```

```
clean:
    rm -rf subdir
```

Аргументы, чтобы сделать

Есть хороший [список вариантов](#), которые можно использовать из макияжа. Проверьте `--dry-run` , `--touch` , `--old-file` .

У вас может быть несколько целей, т.е. `make clean run test` управляет `clean` Цель, тогда `run` , и тогда `test` .

Переменные Пт. 2

Ароматизаторы и модификации

Существует два вкуса переменных:

- Рекурсивный (использование `=`) - ищет переменные только тогда, когда используется команда, *used* а не когда она *определена*.
- просто расширенный (использование `:=`) - как обычное императивное программирование - только те, которые определены до сих пор, расширяются


```
# Recursive variable. This will print "later" below
one = one ${later_variable}
# Simply expanded variable. This will not print "later" below
two := two ${later_variable}

later_variable = later

all:
    echo ${one}
    echo ${two}
```

Просто расширенный (использование `:=`) позволяет приложить к переменной. Рекурсивные определения дадут бесконечную ошибку цикла.

```
one = hello
# one gets defined as a simply expanded variable (:=) and thus can have
one := ${one} there

all:
    echo ${one}
```

`?=` только устанавливает переменные, если они еще не установлены

```
one = hello
one ?= will not be set
two ?= will be set

all:
    echo ${one}
    echo ${two}
```

Пространства в конце линии не разорваны, но те, которые находятся на старте. Чтобы сделать переменную с одним пространством, используйте `$(nullstring)`

```
with_spaces = hello    # with_spaces has many spaces after "hello"
after = ${with_spaces}there

nullstring =
space = ${nullstring} # Make a variable with a single space.

all:
    echo "${after}"
    echo start"${space}"end
```

Неопределенная переменная на самом деле является пустой строкой!

```
all:
    # Undefined variables are just empty strings!
    echo $(nowhere)
```

Использовать += чтобы приложить

```
foo := start
foo += more

all:
    echo $(foo)
```

[Замещение струн](#) также является действительно распространенным и полезным способом изменения переменных. Также проверьте [функции текста](#) и [функции Filename](#).

Аргументы командной строки и переопределение

Вы можете переопределить переменные, которые исходят из командной строки, используя `override`. Здесь мы побежали сделать `make option_one=hi`

```
# Overrides command line arguments
override option_one = did_override
# Does not override command line arguments
option_two = not_override
all:
    echo $(option_one)
    echo $(option_two)
```

Список команд и определение

[Определенная директива](#) не является функцией, хотя она может выглядеть именно так. Я видел, что он используется так редко, что я не буду вдаваться в подробности, но он в основном используется для определения [рецептов консервов](#), а также хорошо сочетается с функцией [эвала](#).

`define / endif` просто создает переменную, которая устанавливается в список команд. Обратите внимание, что это немного отличается от наличия точки за скобками между командами, потому что каждая из них работает в отдельной оболочке, как и ожидалось.

```
one = export blah="I was set!"; echo $$blah

define two
export blah="I was set!"
echo $$blah
endif

all:
```

```
@echo "This prints 'I was set'"
@$(one)
@echo "This does not print 'I was set' because each command runs in
@$(two)
```

Целевые переменные

Переменные могут быть установлены для конкретных целей

```
all: one = cool

all:
    echo one is defined: $(one)

other:
    echo one is nothing: $(one)
```

Картинные переменные

Вы можете установить переменные для конкретных целевых *шаблонов*

```
%.c: one = cool

blah.c:
    echo one is defined: $(one)

other:
    echo one is nothing: $(one)
```

Условная часть Makefiles

Условный if/else

```
foo = ok

all:
    ifeq ($(foo), ok)
        echo "foo equals ok"
    else
        echo "nope"
    endif
```

Проверьте, пуста ли переменная

```
nullstring =
foo = $(nullstring) # end of line; there is a space here
```

```
all:
ifeq ($(strip $(foo)),)
    echo "foo is empty after being stripped"
endif
ifeq ($(nullstring),)
    echo "nullstring doesn't even have spaces"
endif
```

Проверьте, определена ли переменная

ifdef не расширяет переменные ссылки; он просто видит, если что-то вообще определено

```
bar =
foo = $(bar)

all:
ifdef foo
    echo "foo is defined"
endif
ifndef bar
    echo "but bar is not"
endif
```

\$ (MAKEFLAGS)

Этот пример показывает, как проверить флаги с помощью `findstring` и `MAKEFLAGS`. Запустите этот пример с `make -i` Чтобы увидеть, как он распечатывает эхо-заявление.

```
all:
# Search for the "-i" flag. MAKEFLAGS is just a list of single charac
ifneq (,$(findstring i, $(MAKEFLAGS)))
    echo "i was passed to MAKEFLAGS"
endif
```

Функции

Первые функции

Функции предназначены в основном только для обработки текста. Позвонить функции с `$(fn, arguments)` или `${fn, arguments}`. Make имеет приличное количество **встроенных** функций.

```
bar := ${subst not,"totally", "I am not superman"}
all:
```

```
@echo $(bar)
```

Если вы хотите заменить пробелы или запятые, используйте переменные

```
comma := ,
empty:=
space := $(empty) $(empty)
foo := a b c
bar := $(subst $(space),$(comma),$(foo))

all:
  @echo $(bar)
```

Не включайте пробелы в аргументы после первого. Это будет рассматриваться как часть строки.

```
comma := ,
empty:=
space := $(empty) $(empty)
foo := a b c
bar := $(subst $(space), $(comma) , $(foo)) # Watch out!

all:
  # Output is ", a , b , c". Notice the spaces introduced
  @echo $(bar)
```

Замещение строк

`$(patsubst pattern,replacement,text)` делает следующее:

«Находит отдельные белым пространством слова в тексте, которые соответствуют шаблону и заменяют их заменой. Здесь шаблон может содержать «%», который действует как wild card, сопоставляя любое количество любых символов в слове. Если замена также содержит «%», «%» заменяется текстом, который соответствовал «%» в шаблоне. Только первое «%» в шаблоне и замене рассматривается таким образом; любое последующее «%» неизменно. (документы [GNU](#))

Справка о замещении `$(text:pattern=replacement)` Это сокращение для этого.

Есть еще одно сокращение, которое заменяет только суффиксы:

`$(text:suffix=replacement)` . Нет % Здесь используется wild card.

Примечание: не добавляйте дополнительные пространства для этой стенографии. Это будет рассматриваться как термин поиска или замены.

```
foo := a.o b.o l.a c.o
one := $(patsubst %.o,%.c,$(foo))
# This is a shorthand for the above
two := $(foo:%.o=%.c)
# This is the suffix-only shorthand, and is also equivalent to the ak
```

```
three := $(foo:.o=.c)
```

```
all:
    echo $(one)
    echo $(two)
    echo $(three)
```

Каждая функция

Каждая функция выглядит так: `$(foreach var,list,text)` . Он преобразует один список слов (разделенный пространствами) в другой. `var` Устанавливается на каждое слово в списке, и `text` Расширяется для каждого слова.

Это добавляет восклицательный знак после каждого слова:

```
foo := who are you
# For each "word" in foo, output that same word with an exclamation a
bar := $(foreach wrd,$(foo),$(wrd)!)

```

```
all:
    # Output is "who! are! you!"
    @echo $(bar)
```

Функция если

`if` Проверяет, является ли первый аргумент не пустым. Если это так, запускает второй аргумент, в противном случае идет третий.

```
foo := $(if this-is-not-empty,then!,else!)
empty :=
bar := $(if $(empty),then!,else!)
```

```
all:
    @echo $(foo)
    @echo $(bar)
```

Функция вызова

Делайте поддержку, создавая основные функции. Вы «определяете» функцию, просто создавая переменную, но используете параметры `$(0)` , `$(1)` , и т.д. Затем вы называете функцию специальным `call` встроенная функция. Синтаксис является `$(call variable,param,param)` . `$(0)` Это переменная, в то время как `$(1)` , `$(2)` , и т.д. являются параметрами.

```
sweet_new_fn = Variable Name: $(0) First: $(1) Second: $(2) Empty Var
```

```
all:
    # Outputs "Variable Name: sweet_new_fn First: go Second: tigers Empr
    @echo $(call sweet_new_fn, go, tigers)
```

Функция оболочки

раковина - это называется оболочка, но она заменяет новые линии пространствами!

```
all:
    @echo $(shell ls -la) # Very ugly because the newlines are gone!
```

Функция фильтра

The `filter` Функция используется для выбора определенных элементов из списка, которые соответствуют определенному шаблону. Например, это позволит выбрать все элементы в `obj_files` В этом конце с `.o` .

```
obj_files = foo.result bar.o lose.o
filtered_files = $(filter %.o,$(obj_files))

all:
    @echo $(filtered_files)
```

Фильтр также может использоваться более сложными способами:

1. **Фильтрация нескольких шаблонов:** вы можете фильтровать несколько шаблонов одновременно. Например, например, `$(filter %.c %.h, $(files))` будет выбрать все `.c` и `.h` Файлы из списка файлов.
2. **Negation**Отрицание: Если вы хотите выбрать все элементы, которые не соответствуют шаблону, вы можете использовать `filter-out` . Например, например, `$(filter-out %.h, $(files))` выберет все файлы, которые не являются `.h` Файлы.
3. **Вложенный фильтр:** Вы можете гнездить функции фильтра для применения нескольких фильтров. Например, например, `$(filter %.o, $(filter-out test%, $(objects)))` будет выбирать все файлы объектов, которые заканчиваются `.o` но не начинайте с `test` .

Другие особенности

Включите Makefiles

Включая директиву, в которой говорится, что нужно прочитать один или несколько других файлов. Это строка в макияже, которая выглядит так:

```
include filenames...
```

Это особенно полезно, когда вы используете флаги компилятора, такие как `-M` которые создают Makefiles на основе источника. Например, если некоторые файлы с включают заголовок, этот заголовок будет добавлен в Makefile, который

написан gcc. Я рассказываю об этом больше в [Makefile Cookbook](#)

Директива vpath

Используйте путь, чтобы указать, где существует набор предпосылок. Формат является `vpath <pattern> <directories, space/colon separated>` `<pattern>` может иметь `%`, который соответствует любому нулю или более персонажам. Вы также можете сделать это глобально с переменной `VPATH`

```
vpath %.h ../headers ../other-directory
```

```
# Note: vpath allows blah.h to be found even though blah.h is never i
some_binary: ../headers blah.h
    touch some_binary
```

```
../headers:
    mkdir ../headers
```

```
# We call the target blah.h instead of ../headers/blah.h, because tha
# Typically, blah.h would already exist and you wouldn't need this.
blah.h:
    touch ../headers/blah.h
```

```
clean:
    rm -rf ../headers
    rm -f some_binary
```

Многолинейный

Характер обратных ударов («\») дает нам возможность использовать несколько строк, когда команды слишком длинные.

```
some_file:
    echo This line is too long, so \
        it is broken up into multiple lines
```

.phony

Добавить `.PHONY` для цели не позволит Make перепутать фальшивую цель с именем файла. В этом примере, если файл `clean` Создается, сделать чистоту все-таки будет запущено. Технически, я должен был использовать его в каждом примере с `all` или `clean` Но я хотел сохранить примеры чистыми. Кроме того, «фальшивые» мишени обычно имеют имена, которые редко являются именами файлов, и на практике многие люди пропускают это.

```
some_file:
    touch some_file
    touch clean
```



```
.PHONY: clean
clean:
    rm -f some_file
    rm -f clean
```

.delete_on_error

Инструмент Make перестанет запускать правило (и будет продвигаться обратно к предпосылкам), если команда возвращает статус ненулевое выход.

DELETE_ON_ERROR будет исключить цель правила, если правило не работает таким образом. Это произойдет для всех целей, а не только для тех, которые раньше были, как PHONY. Это хорошая идея, чтобы всегда использовать это, хотя сделать не по историческим причинам.

```
.DELETE_ON_ERROR:
all: one two
```

```
one:
    touch one
    false
```

```
two:
    touch two
    false
```

Макифайл Кулинарная Книга

Давайте пройдемся по действительно сочному примеру, который хорошо подходит для проектов среднего размера.

Самое замечательное в этом макияже заключается в том, что он автоматически определяет зависимости для вас. Все, что вам нужно сделать, это поместить ваши файлы C/C++ в `src/` Папка.

```
# Thanks to Job Vranish (https://spin.atomicobject.com/2016/08/26/makefile-recipe/)
TARGET_EXEC := final_program
```

```
BUILD_DIR := ./build
SRC_DIRS := ./src
```

```
# Find all the C and C++ files we want to compile
# Note the single quotes around the * expressions. The shell will inc
SRCS := $(shell find $(SRC_DIRS) -name '*.cpp' -or -name '*.c' -or -r
```

```
# Prepends BUILD_DIR and appends .o to every src file
# As an example, ./your_dir/hello.cpp turns into ./build/./your_dir/h
OBJS := $(SRCS:%=$(BUILD_DIR)/%.o)
```

```
# String substitution (suffix version without %).
# As an example, ./build/hello.cpp.o turns into ./build/hello.cpp.d
DEPS := $(OBJS:.o=.d)

# Every folder in ./src will need to be passed to GCC so that it can
INC_DIRS := $(shell find $(SRC_DIRS) -type d)
# Add a prefix to INC_DIRS. So moduleA would become -I moduleA. GCC ur
INC_FLAGS := $(addprefix -I,$(INC_DIRS))

# The -MMD and -MP flags together generate Makefiles for us!
# These files will have .d instead of .o as the output.
CPPFLAGS := $(INC_FLAGS) -MMD -MP

# The final build step.
$(BUILD_DIR)/$(TARGET_EXEC): $(OBJS)
    $(CXX) $(OBJS) -o $@ $(LDFLAGS)

# Build step for C source
$(BUILD_DIR)/%.c.o: %.c
    mkdir -p $(dir $@)
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

# Build step for C++ source
$(BUILD_DIR)/%.cpp.o: %.cpp
    mkdir -p $(dir $@)
    $(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -r $(BUILD_DIR)

# Include the .d makefiles. The - at the front suppresses the errors
# Makefiles. Initially, all the .d files will be missing, and we don't
# errors to show up.
-include $(DEPS)
```